

## অধ্যায়-২

## অ্যালগরিদম

**SOS**

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

**\*\*\* ১। Algorithm বলতে কী বুঝায়?**

ICT Ministry- 2017,

বাকাশিবো- ২০০৯, ০৯' পরি, ১১, ১৩' পরি, ১৫' পরি, ১৮, ১৯, ২২

**উত্তর :** কোনো সমস্যা সমাধানের যুক্তিসম্মত ও পর্যায়ক্রমিক ধারার প্রক্রিয়াকে Algorithm বলে। Algorithm এর সর্ব প্রথম ধারণা দেন আরবি গণিতবিদ মুহাম্মদ ইবনে মুসা-আল-খোয়ারিজমী।

**\*\*\* ২। Pseudo Code বা সুডো কোড বলতে কী বুঝায়?**

বাকাশিবো- ০৩, ০৬, ০৬' পরি, ০৮, ০৯, ০৯' পরি, ১০, ১১, ১২, ১৪' পরি, ১৫' পরি, ১৮, ২০

**উত্তর :** Pseudo শব্দের অর্থ ছদ্ম বা সত্য নয় এমন। অনেক সময় প্রোগ্রাম লেখার আগে প্রোগ্রামের একটি খসড়া তৈরি করা হয়, যা দেখতে অনেকটা প্রোগ্রামিং কোডের মত কিন্তু এটি আসলে কোড নয়। প্রোগ্রামের ধরন ও কার্যাবলী তুলে ধরার জন্য কিছু সংখ্যক নির্দেশনার সমষ্টিকে Pseudo Code (সুডো কোড) বা Program Design Language (PDL) বলে।

### \*\*\* ৩। Algorithm Notation কী?

বাকশিবো- ২০০৬, ০৭, ০৮, ০৯, ১২, ১৫' পরি, ১৬' পরি, ১৮' পরি

**উত্তর :** Algorithm এ বিধিবদ্ধ কিছু সংকেত ব্যবহার করা হয়। Algorithm এর এই বিধিবদ্ধ সকল সংকেতকে Algorithm Notation বলে।

যেমন : Big-0 Notation (O-Notation)

Omega Notation ( $\Omega$  – Notation)

Theta Notation ( $\theta$  – Notation)

### \*\* ৪। Flow Chart বা প্রবাহ চিত্র কী?

বাকশিবো- ২০০০, ০২

**উত্তর :** যে চিত্রের মাধ্যমে কোন প্রোগ্রাম কীভাবে কাজ করবে তার গতিধারা নির্ধারণ করা হয়, তাকে Flowchart বা প্রবাহচিত্র বলে।

অর্থাৎ Algorithm এর চিত্রভিত্তিক রূপকে Flowchart বলে।

### \*\* ৫। Bug ও Debugging কী?

বাকশিবো- ২০০৬, ১১, ১৭

**উত্তর :** Bug: Program এর মধ্যে যত ধরনের ত্রুটি আছে, তাই Bug.

Debug: Source Code এর ত্রুটি সংশোধন করাই হচ্ছে Debug.

### \* ৬। Algorithm এর Complexity নির্ণয়ের বিবেচ্য বিষয়গুলো কী কী?

বাকশিবো- ২০১৬, ২৩

**উত্তর :** Algorithm এর Complexity নির্ণয়ের বিবেচ্য বিষয় তিনটি।

যথা : i) Worst Case    ii) Average Case    iii) Best Case.

**\*\*\* ৭। Structure Programming বলতে কী বুঝায়?**

বাকশিবো- ২০০৪, ০৭, ০৯, ০৯' পরি, ১৫' পরি, ২২

**উত্তর :** Structure Programming হলো Logical programming পদ্ধতি, যা Object Oriented Programming (OOP) এর পূর্বের Programming Structure. এই সকল প্রোগ্রাম Top-Down Design পদ্ধতিতে কাজ করে। এই সকল প্রোগ্রামিংকে Modular Programming ও বলা হয়।

যেমন : C, ALGOL, Pascal ইত্যাদি।

**SOS****সংক্ষিপ্ত প্রশ্নোত্তর :****\*\*\* ১। Algorithm এর বৈশিষ্ট্যগুলি লেখ।**

বাকশিবো- ২০০৬, ০৭, ১৪' পরি, ১৬, ১৮, ১৯, ১৯' পরি, ২৩

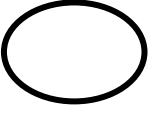
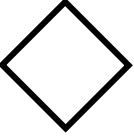
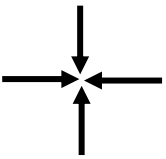
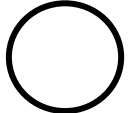
**উত্তর :** Algorithm এর বৈশিষ্ট্যগুলি নিম্নরূপ :

১. প্রতিটি Algorithm এর শুরু এবং শেষ থাকবে।
২. প্রতিটি Algorithm এর সুনির্দিষ্ট কতগুলো Step বা ধাপ থাকবে।
৩. প্রতিটি Algorithm এর এক বা একাধিক Input থাকবে।
৪. প্রতিটি Algorithm এর সুনির্দিষ্ট Output থাকবে।
৫. প্রতিটি ধাপ সহজবোধ্য হতে হবে।
৬. প্রতিটি ধাপের মাঝে পারস্পরিক সম্পর্ক থাকতে হবে এবং
৭. প্রতিটি ধাপের কাজ যুক্তি নির্ভর হতে হবে।

\*\*\* ২। **Flow-chart** এর বিভিন্ন প্রতীকগুলির নাম ও কাজ লেখ।

বাকাশিবো- ২০০৯' পরি, ১২, ১২' পরি, ১৫' পরি

**উত্তর :** Flow-chart এর বিভিন্ন প্রতীকগুলির নাম ও কাজ নিম্নরূপ :

| প্রতীক                                                                              | নাম                                | কাজ                                                                |
|-------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------|
|    | Start/End                          | Flowchart শুরু ও শেষ করার জন্য এই চিহ্ন ব্যবহৃত হয়।               |
|    | Input/Output                       | Input এবং Output নেওয়ার জন্য এই চিহ্ন ব্যবহার করা হয়।            |
|    | Process বা প্রক্রিয়াকরণ           | গাণিতিক সকল Expression এ এই চিহ্ন ব্যবহার করা হয়।                 |
|  | Decision Making বা সিদ্ধান্ত গ্রহণ | শর্তাধীন কোনো কাজ করার জন্য এই চিহ্ন ব্যবহার করা হয়।              |
|  | Flow বা নির্দেশনা                  | Program Flow কোন দিকে যাচ্ছে তা বুঝানোর জন্য এই চিহ্ন ব্যবহৃত হয়। |
|  | Connection বা সংযোগ                | একাধিক অংশকে সংযুক্ত করার জন্য এই চিহ্ন ব্যবহৃত হয়।               |
|  | Print বা Document                  | Program এর কোনো অংশ Print করার দরকার হলে এই চিহ্ন ব্যবহৃত হয়।     |
|  | Loop                               | একই কাজ বারবার (Looping) করার জন্য ব্যবহৃত হয়।                    |

**\*\* ৩। Algorithm ও Flowchart এর মাঝে পার্থক্য লেখ।**

বাকাশিবো- ২০০৯'পরি

**উত্তর :** Algorithm ও Flowchart এর মাঝে পার্থক্য নিম্নরূপ :

| Algorithm                                                                                  | Flowchart                                          |
|--------------------------------------------------------------------------------------------|----------------------------------------------------|
| ১) নির্দিষ্ট সমস্যা সমাধানের জন্য যুক্তিসম্মত ও ধাপে ধাপে সমাধানের পদ্ধতিকে Algorithm বলে। | ১) Algorithm এর চিত্রভিত্তিক রূপকে Flowchart বলে।  |
| ২) এটি বর্ণনামূলক।                                                                         | ২) এটি চিত্রনির্ভর।                                |
| ৩) Algorithm এর ভিত্তিতে Flowchart তৈরি করা হয়।                                           | ৩) Flowchart এর ভিত্তিতে Program রচনা করা হয়।     |
| ৪) এতে কোনো চিত্র ব্যবহৃত হয় না।                                                          | ৪) এতে চিত্র ব্যবহৃত হয়।                          |
| ৫) Algorithm তৈরির পূর্বে সুডো কোড তৈরির প্রয়োজন।                                         | ৫) Flowchart তৈরির পূর্বে Algorithm লেখা প্রয়োজন। |

[**Note:** সুডো কোড : Program এর ধরন এবং কার্যপ্রণালী সংবলিত কিছু নির্দেশকেই সুডো বা Pseudo Code বলে।]

## \*\* ৪। ফারেনহাইট তাপমাত্রাকে সেন্টিগ্রেড এ রূপান্তরের Algorithm ও Flowchart আঁক।

বাকাশিবো- ২০১৩' পরি, ১৭' পরি

**উত্তর :** ফারেনহাইট তাপমাত্রাকে সেন্টিগ্রেড এ রূপান্তরের Algorithm ও Flowchart নিম্নরূপ :

### Algorithm

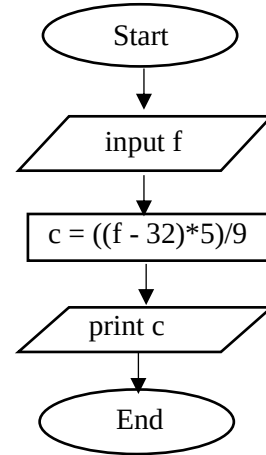
Step 1: Start

Step 2: input f

Step 3:  $c = ((f - 32) * 5) / 9$

Step 4: print c

Step 5: End



### Flowchart

তাপমাত্রার স্কেল

পাঠ নিম্ন - স্থিরাক্ষ

উর্ধ্ব স্থিরাক্ষ - নিম্ন স্থিরাক্ষ

$$\text{বা, } \frac{C-0}{100-0} = \frac{F-32}{212-32} = \frac{K-273}{373-273} = \frac{R-0}{80-0}$$

$$\text{বা, } \frac{C}{100} = \frac{F-32}{180} = \frac{K-273}{100} = \frac{R}{80}$$

$$\text{বা, } \frac{C}{5} = \frac{F-32}{9} = \frac{K-273}{5} = \frac{R}{4}$$

|     |     |     |    |
|-----|-----|-----|----|
| 100 | 212 | 373 | 80 |
|     |     |     |    |
| 0   | 32  | 273 | 0  |
| C   | F   | K   | R  |

## ❖ Fibonacci Series ❖

**Fibonacci Series :** Fibonacci Series এর আবিষ্কারক ত্রয়োদশ শতাব্দীর বিখ্যাত গণিতবিদ ‘Leonardo Da Piso’ উনার ডাকনাম ছিল Fibonacci. এ সিরিজটি শুরু হয় 0 থেকে এবং সিরিজের পরবর্তী সংখ্যাগুলো প্রতিটি তার পূর্বের দুইটি পদের যোগফল।

যেমন : 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89.....

**Fibonacci Series এর বৈশিষ্ট্য :**

- i) যেকোনো সংখ্যা তার আগের দুটি রাশির যোগফলের সমান।
- ii) যেকোনো সংখ্যা তার পরের দুটি রাশির বিয়োগফলের সমান।
- iii) যেকোনো সংখ্যাকে তার আগের সংখ্যা দিয়ে ভাগ করলে Golden Ratio (1.618) পাওয়া যায়, (0, 1, 2) বাদে।

**বাস্তব উদাহরণ :**

- i) পাখির ঝাঁক বেধে আকাশে উড়া।
- ii) সূর্যমুখী ফুলের পাপড়ির বিন্যাস।
- iii) শামুকের প্যাচ।
- iv) ফুলকপির বিন্যাস।
- v) মৌমাছির পরিবার তন্ত্র।
- vi) মহাকাশের ছায়াপথ।
- vii) বেশিরভাগ ফুলের পাপড়ির বিন্যাস ইত্যাদি।

\*\*\* ৫। Fibonacci Series বের করার Algorithm লেখ।

বাকশিবো- ২০০৬, ০৬' পরি, ০৯, ১২' পরি, ১৬' পরি, ২২

**উত্তর :** Algorithm :

Step 1 : যদি  $n=0$  অথবা  $n=1$  হয়, তবে,  $FIB=n$  এবং Return.

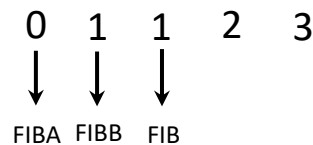
Step 2 : Call FIBONACCI (FIBA, N-1)

Step 3 : Call FIBONACCI (FIBB, N-2)

Step 4 : Set  $FIB = FIB A + FIB B$

Step 5 : Return

ব্যাখ্যা :



[ **Note:** পরীক্ষাতে উত্তর হিসেবে শুধু Algorithm লিখলেই হবে]

\*\*\* ৬। Factorial বের করার Algorithm লেখ।

বাকশিবো- ২০১৪' পরি, ১৭' পরি, ১৯

**উত্তর :** Factorial বের করার Algorithm নিম্নরূপ :

**Algorithm :**

Step 1 : যদি  $N=0$  তবে, Set  $FACT=1$  এবং Return করবে।

Step 2 : Call FACTORIAL (FACT, N-1)

Step 3 : Set  $FACT = K * FACT$

Step 4 : Return.



ব্যাখ্যা :  $n! = n(n-1)! = n(n-1)(n-2)! = n(n-1)(n-2) \dots \dots \dots (n-n)!$

উদাহরণ :  $4!$

$$= 4 \times 3!$$

$$3 \times 2!$$

$$2 \times 1!$$

$$1 \times 0!$$

$$1 \times 1$$

$$2 \times 1$$

$$3 \times 2$$

$$4 \times 6$$

### \*\*\* ৭। Algorithm Complexity বলতে কী বুঝ?

বাকশিবো- ২০০৫, ০৯, ০৯' পরি, ১১, ১২, ১৪, ১৪' পরি, ১৫' পরি, ১৯' পরি

BCS- 35th, 36th, কারিগরি শিক্ষা অধিদপ্তর- ২০১৯

**উত্তর :** যখন কোনো Algorithm ডিজাইন করা হয়, তার আগে জানা প্রয়োজন Algorithm টি কতটা কার্যকর। সেক্ষেত্রে নিচের তিনটি বিষয় মাথায় রাখতে হবে :

- Algorithm টি কী নির্ধারিত সময়ের মধ্যে ফলাফল দিবে?
- সর্বোচ্চ কত বড় বা কতগুলি মানের জন্য এটি কাজ করবে?
- কতটুকু মেমরি ব্যবহার করবে?

Algorithm Complexity এর মাধ্যমে প্রথম দুটি বিষয়ের উত্তর পাওয়া যায়। একটি Algorithm যতগুলো Instruction ব্যবহার করে

কাজ করে তাই Algorithm এর Complexity. Complexity থেকে বুঝা যায় Algorithm টি তুলনামূলকভাবে কতটা ভালো।

Algorithm Complexity দুই ধরনের। যথা :

**i) Time Complexity :** একটি প্রোগ্রাম রান হতে কতক্ষণ সময় লাগে, তাকে Time Complexity বলে।

**ii) Space Complexity :** একটি প্রোগ্রাম মেমরিতে কতটুকু জায়গা নেয়, তাকে Space Complexity বলে।

Algorithm Complexity তিনটি বিষয়ের উপর নির্ভরশীল।

**i) Worst Case :** Running Time ও Space সবচেয়ে বেশি।

**ii) Average Case :** Running Time ও Space গড় মানের।

**iii) Best Case :** Running Time ও Space সবচেয়ে কম।

**SOS****রচনামূলক প্রশ্নোত্তর :**

\*\*\* ১। তিনটি সংখ্যার মাঝে ক্ষুদ্রতম বা বৃহত্তম সংখ্যা নির্ণয়ের জন্য একটি অ্যালগরিদম ও ফ্লোচার্ট অঙ্কন কর।

বাকশিবো- ২০০৯, ০৯' পরি, ১৫' পরি, ১৬, ১৮, ১৮' পরি, ২২

**উত্তর :****Algorithm**

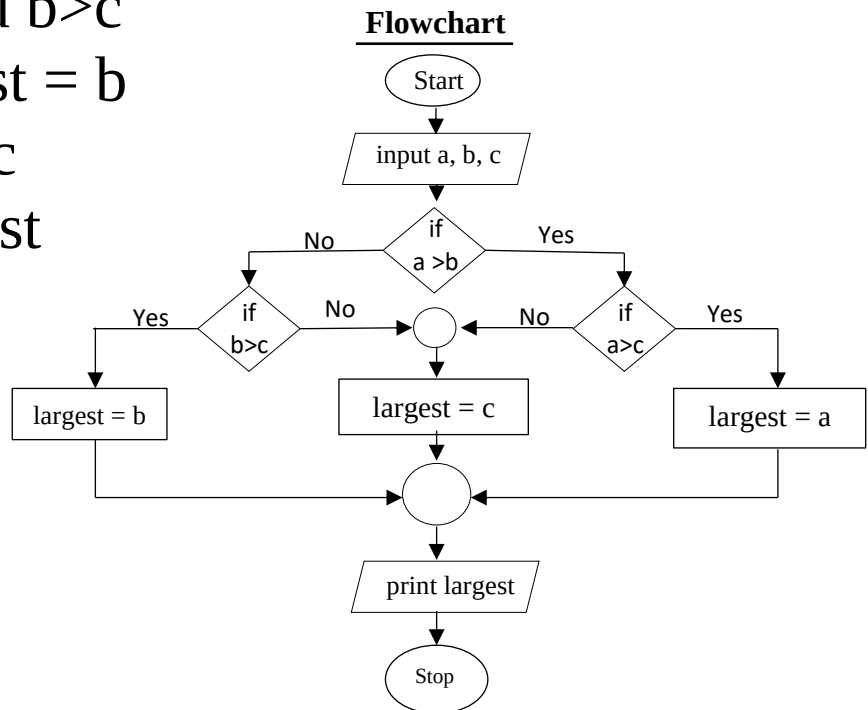
Step 1: Start

Step 2: input three numbers a, b, c

Step 3: if  $a > b$  and  $a > c$   
 then largest = a  
 else if  $b > a$  and  $b > c$   
 then largest = b  
 else largest = c

Step 4: print largest

Step 5: Stop



**Note:** ছোট সংখ্যা নির্ণয় করতে বললে Greater than ( $>$ ) এর পরিবর্তে less Than ( $<$ )

\*\*\* ২। দ্বিঘাত সমীকরণ বা Quadratic equation ( $ax^2 + bx + c = 0$ ) এর মূল বের করার Algorithm ও Flowchart

আঁক। বাকাশিবো- ২০০০, ০২, ০৫, ০৮, ১০, ১২, ১২' পরি, ১৫' পরি, ১৮' পরি, ১৯, ২২

উত্তর :

### Algorithm

Step 1: Start

Step 2: input a, b, c

Step 3: নিশ্চায়ক  $D = b^2 - 4ac$

Step 4: if  $D > 0$  then

$$(i) x_1 = \frac{-b + \sqrt{D}}{2a}$$

$$(ii) x_2 = \frac{-b - \sqrt{D}}{2a}$$

print  $x_1, x_2$

else if  $D = 0$  then

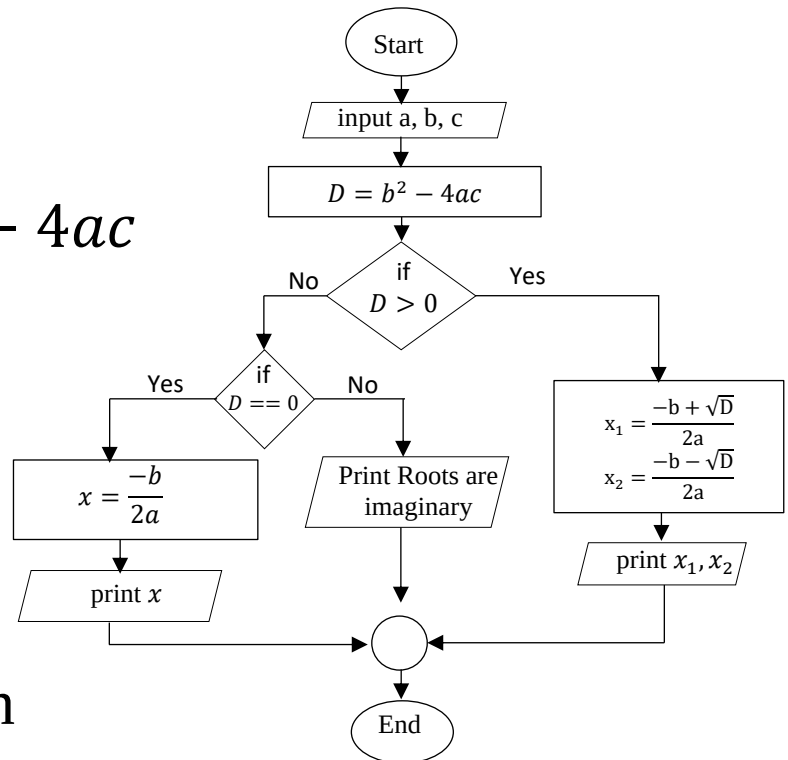
$$x = \frac{-b}{2a}$$

print  $x$

else roots are imaginary

Step 5: End

### Flowchart



\*\*\* ৩। ত্রিভুজের তিন বাহুর সাহায্যে ত্রিভুজের ক্ষেত্রফল নির্ণয়ের

## Algorithm ও Flowchart আঁক।

**উত্তর :** শর্ত : যে কোনো দুই বাহুর যোগফল অবশ্যই তার তৃতীয় বাহু অপেক্ষা বড় হতে হবে। অন্যথায় ত্রিভুজ গঠন করা যাবে না।

অর্থাৎ,  $(a + b) > c$  এবং  $(b + c) > a$  এবং  $(a + c) > b$  সবগুলি শর্তই সত্য হতে হবে।

### Algorithm

Step 1: Start

Step 2: input  $a, b, c$

Step 3: if  $(a + b) > c$  and  $(b + c) > a$  and  $(a + c) > b$  then

(i)  $S = (a + b + c)/2$

(ii)  $area = \sqrt{S \times (S - a) \times (S - b) \times (S - c)}$

print area

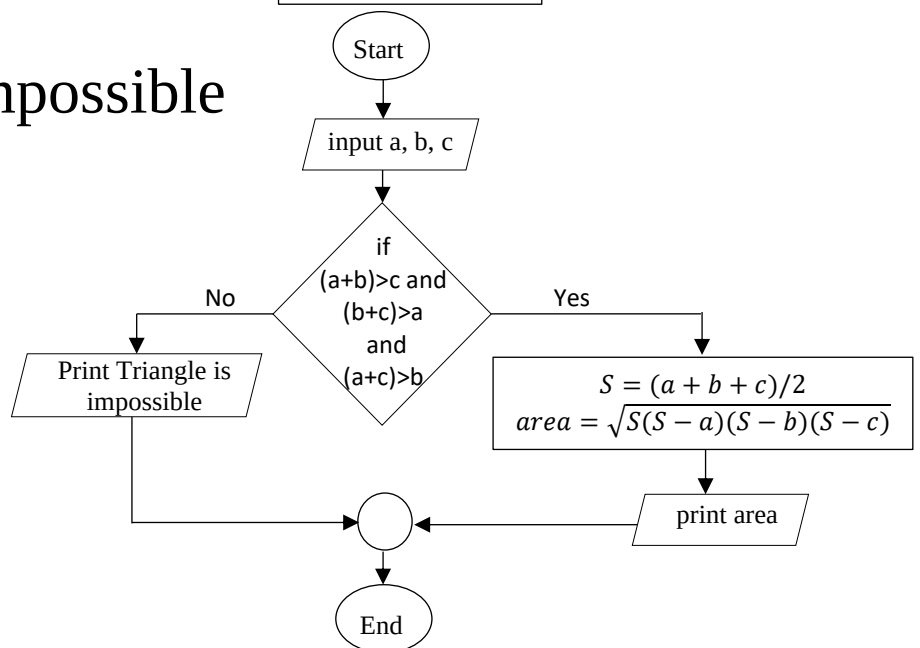
else

print Triangle is impossible

Step 4: Stop

$2S =$  পরিসীমা (সবগুলো বাহুর যোগফল)  
 $S =$  অর্ধপরিসীমা

### Flowchart



\*\*\* ৪। প্রোগ্রাম পরিকল্পনার ধাপসমূহ বর্ণনা কর।

বাকশিৰো- ২০০২, ১০, ১২' পরি, ১৩' পরি

**উত্তর :** প্রোগ্রাম দিয়ে নির্দিষ্ট কোনো সমস্যা সমাধান করতে হলে অবশ্যই সুনির্দিষ্ট কতগুলো ধাপে তা সম্পন্ন করতে হয়। ইহাকে প্রোগ্রাম পরিকল্পনার ধাপ বা Program Planning বলে। কারণ সুনির্দিষ্ট পরিকল্পনা ছাড়া কোনো কাজই সুসম্পন্ন হয় না।

প্রোগ্রাম পরিকল্পনার ধাপসমূহ নিম্নরূপ :

১) সমস্যা চিহ্নিত করা (**Problem Identification**) :- এটি প্রোগ্রাম পরিকল্পনার সর্বপ্রথম ধাপ। কারণ সবার প্রথমে আমি কি করতে চাই তা নির্ধারণ করতে হবে।

২) সমস্যা বিশ্লেষণ (**Problem Analysis**) : এই ধাপে সমস্যাকে বিশ্লেষণ করে সবচেয়ে সহজ উপায়ে সমস্যার সমাধান নির্ধারণ করা হয়।

৩) প্রোগ্রামের ইনপুট/আউটপুট নির্ণয় : এই ধাপে প্রয়োজনীয় Input ও Output নির্ধারণ করে কাক্সিত Output পাওয়ার জন্য সকল সূত্রের প্রয়োগ করতে হয়।

৪) প্রোগ্রামের অ্যালগরিদম উন্নয়ন করা (**Algorithm Development**) : Program টি কিভাবে কাজ করবে তা Algorithm ধাপে উল্লেখ করা থাকে। Program লেখার আগে তা কিভাবে কাজ করবে সেই সম্পর্কে অবশ্যই স্বচ্ছ ধারণা থাকতে হবে।

৫) ফ্লোচার্ট তৈরি করা (**Flowchart Development**) : Algorithm এর চিত্রভিত্তিক রূপই হলো Flowchart। এর জন্য নির্দিষ্ট কিছু চিত্র আছে, যা Program শুরু করার আগে Algorithm অনুযায়ী তৈরি করতে হবে।

৬) প্রোগ্রামিং ভাষা নির্ধারণ : অনেক প্রোগ্রামিং ভাষা থেকে আমার সমস্যার ধরন অনুযায়ী কোন ভাষা ব্যবহার করা বেশি সুবিধাজনক তা নির্ধারণ করতে হবে।

৭) প্রোগ্রাম রচনা করা (**Program Coding**) : এই ধাপে যে প্রোগ্রামিং ল্যাংগুয়েজ নির্ধারণ করা হয়েছে সেই অনুযায়ী Program লেখা শুরু করতে হবে।

৮) প্রোগ্রাম কম্পাইল করা : High Level Language কে Machine Language এ রূপান্তরের জন্য Compiler দ্বারা Program কে Compile করা হয়। ফলে যদি Program এ কোনো ভুল থাকে তবে তা ধরা পড়ে।

৯) প্রোগ্রাম পরীক্ষণ ও সংশোধন (**Program Testing And Debugging**) : Compile করার পর যে ভুলগুলি থাকে তাকে Bug বলে এবং এই Bug দূর করা বা সংশোধন করাকে Debugging বলে এবং বিভিন্ন ধরনের Testing (Alpha Test, Beta Test) ইত্যাদি করা হয়।

১০) **Documentation** : এই ধাপে সকল কার্যক্রমকে ভবিষ্যতের জন্য সংগ্রহ করে রাখা হয়। যেন পরবর্তীতে প্রয়োজন অনুযায়ী তা আবার ব্যবহার বা পরিবর্তন করা যায়।

১১) **Installation** : সকল Testing সম্পন্ন হবার পর প্রোগ্রামসমূহকে প্রোডাকশন লাইব্রেরীতে Load করা হয়। একে Installation বলে।

১২) রক্ষণাবেক্ষণ (**Maintenance**) : এটি প্রোগ্রাম পরিকল্পনার সর্বশেষ ধাপ। এতে ছোট কোনো পরিবর্তন বা পরিবর্ধন করা বুঝায়। কারণ একটি Software যখন বাজারে আসে তাতে নতুন কোনো আপডেট আসতেই পারে। ফলে তার উন্নত Version বাজারে আনতে হয়।

[**Note:** এই প্রশ্ন সংক্ষিপ্ত আকারে আসলে শুধু মাত্র ধাপ গুলো লিখলেই হবে]

**\*\*\* ৫। Algorithm এর Complexity বর্ণনা কর।**

বাকশিবো- ২০০২, ০৬, ১৩'পরি, ১৫, ১৭'পরি, ১৮'পরি

**উত্তর :** **Algorithm :** কোনো সমস্যা সমাধানের যুক্তিসম্মত ও পর্যায়ক্রমিক ধারার প্রক্রিয়াকে Algorithm বলে। Algorithm এর সর্ব প্রথম ধারণা দেন আরবি গণিতবিদ মুহাম্মদ ইবনে মুসা-আল-খোয়ারিজমী।

**Algorithm Complexity:** যখন কোনো Algorithm ডিজাইন করা হয়, তার আগে জানা প্রয়োজন Algorithm টি কতটা কার্যকর।

সেক্ষেত্রে নিচের তিনটি বিষয় মাথায় রাখতে হবে :

- i) Algorithm টি কী নির্ধারিত সময়ের মধ্যে ফলাফল দিবে?
- ii) সর্বোচ্চ কত বড় বা কতগুলি মানের জন্য এটি কাজ করবে?
- iii) কতটুকু মেমরি ব্যবহার করবে?

Algorithm Complexity এর মাধ্যমে প্রথম দুটি বিষয়ের উত্তর পাওয়া যায়। একটি Algorithm যতগুলো Instruction ব্যবহার করে কাজ করে তাই Algorithm এর Complexity. এটি থেকে বুঝা যায় Algorithm টি তুলনামূলকভাবে কতটা ভালো।

Algorithm Complexity দুই ধরনের। যথা :

**i) Time Complexity :** একটি প্রোগ্রাম রান হতে কতক্ষণ সময় লাগে, তাকে Time Complexity বলে।

**ii) Space Complexity :** একটি প্রোগ্রাম মেমরিতে কতটুকু জায়গা নেয়, তাকে Space Complexity বলে।



**Algorithm Complexity** তিনটি বিষয়ের উপর নির্ভরশীল। যথা :

- i) Worst Case** : Running Time ও Space সবচেয়ে বেশি।
- ii) Average Case** : Running Time ও Space গড় মানের।
- iii) Best Case** : Running Time ও Space সবচেয়ে কম।

Algorithm এ বিধিবদ্ধ কিছু সংকেত ব্যবহার করা হয়। Algorithm এর এই বিধিবদ্ধ সকল সংকেতকে Algorithm Notation বলে।

যেমন : Big-0 Notation ( $O$ -Notation)

Omega Notation ( $\Omega$  – Notation)

Theta Notation ( $\theta$  – Notation)